

Grundlagen von Datenbanken

Datenbanken

Lernzettel

Kompakte Zusammenfassung der Vorlesungsinhalte für die Prüfungsvorbereitung.

Autor Levin Rüßmann
Matrikelnummer 838791
Studiengang Informatik B.Sc.
Semester Sommersemester 2026
Datum 21. Juni 2026

Inhaltsverzeichnis

1	Datenbanken — Grundlagen	2
1.1	Datenbank – Definition	2
1.2	Datenbank-Management-System – DBMS	2
1.2.1	Anforderungen an ein DBMS	2
1.2.2	Aufbau eines DBMS	3
2	Datenbankmodelle	4
2.1	Relationales Datenbankmodell	4
2.2	Hierarchisches Datenbankmodell	5
2.3	Netzwerkmodell	6
2.4	Objektorientiertes Datenbankmodell	7
3	Datenverteilung	8
3.1	Gründe für die Datenverteilung	8
4	Datenbank-Architektur	8
4.1	ANSI-3-Ebenen-Modell	9
4.1.1	Interne Ebene	9
4.1.2	Konzeptionelle Ebene	9
4.1.3	Externe Ebene	10
4.2	Datenbank Entwurfs Phasen	10
5	Managementunterstützungssysteme	10
5.1	Data Warehouse	10
5.2	Data Mart	11
5.3	Online Analytical Processing – OLAP	12
5.4	Data Mining	13

1 Datenbanken — Grundlagen

1.1 Datenbank – Definition

Eine Datenbank speichert eine Vielzahl von Daten nach einem bestimmten Datenbankmodell. Im Gegensatz zu einem einfachen Verzeichnis werden die Daten strukturiert und redundanzfrei gespeichert. Außerdem ermöglicht eine Datenbank die dauerhafte (persistente) Speicherung von Daten.

1.2 Datenbank-Management-System – DBMS

Eine Datenbank wird immer durch ein Datenbank-Management-System (DBMS) verwaltet. Beispiele dafür sind MySQL, PostgreSQL oder Microsoft SQL Server. Das DBMS ist die Schnittstelle zur Datenbank – egal ob per CLI oder über eine GUI. Außerdem ermöglicht ein DBMS den Mehrnutzerbetrieb.

Vorteile

- Redundanzfreiheit – Daten werden nur einmal gespeichert
- Flexibilität – Struktur lässt sich nachträglich anpassen
- Physische Datenunabhängigkeit – Anwendung ist von der Speicherform getrennt
- Mehrnutzerbetrieb – mehrere Nutzer können gleichzeitig zugreifen
- Datenintegrität – Konsistenz wird durch das DBMS sichergestellt
- Recovery – Wiederherstellung nach einem Fehler oder Absturz
- Zugriffskontrolle – Rechte können nutzerspezifisch vergeben werden

Nachteile

- Ressourcenintensiv – benötigt mehr Rechenleistung und Speicher
- Komplexität – Einrichtung und Administration erfordern Fachwissen
- Kosten – Anschaffung, Lizenzen und Wartung verursachen laufende Ausgaben
- Schulungsaufwand – Mitarbeiter müssen im Umgang mit dem DBMS geschult werden
- Single Point of Failure – fällt das DBMS aus, sind alle Daten nicht verfügbar

1.2.1 Anforderungen an ein DBMS

1. **Redundanzabbau:** Durch Normalisierung verwaltet das DBMS die Daten so, dass sie möglichst wenig redundant gespeichert werden.
2. **Datenintegrität:** Die gespeicherten Daten müssen korrekt und zuverlässig sein.
3. **Effizienz und Performance:** Daten sollen schnell abgefragt werden können.
4. **Benutzerfreundlichkeit:** Tabellen sollten verständlich benannt sein, damit Daten leicht zu finden sind und in sinnvollen Datentypen vorliegen.
5. **Mehrfachzugriff:** Mehrere Nutzer sollen gleichzeitig auf die Datenbank zugreifen können.
6. **Datenschutz:** Daten werden durch Nutzerrollen und Anonymisierung geschützt.
7. **Datensicherheit:** Daten werden durch Backups gesichert und geschützt.
8. **Datenunabhängigkeit:** Grad, in dem Nutzer und Anwendung unabhängig von der

konkreten Speicherform auf das DBMS zugreifen können.

1.2.2 Aufbau eines DBMS

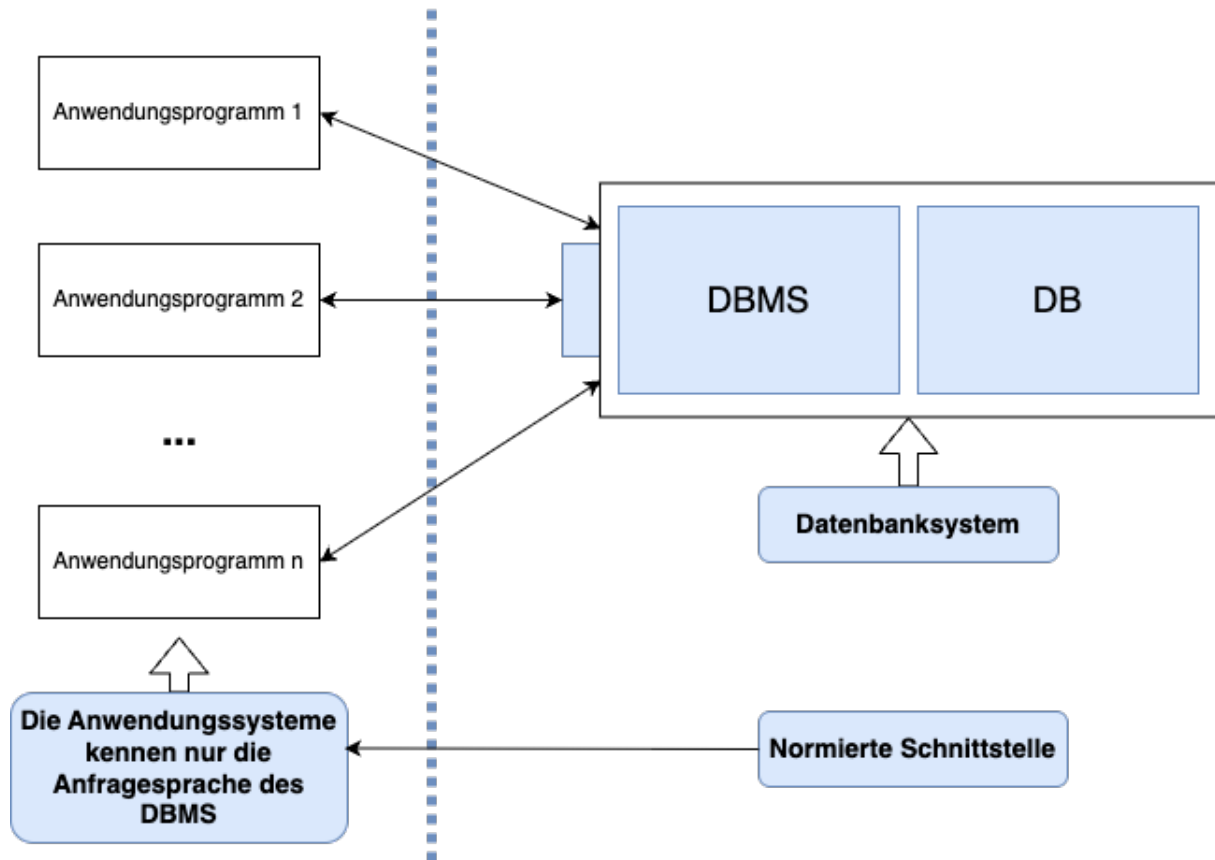


Abbildung 1. Aufbau eines DBMS

Auch bei dem Aufbau kann man Elemente wie den Mehrnutserzugriff erkennen. Auch die Datenunabhängigkeit wird gezeigt, da die Anwendung nur die Anfragesprache kennt, aber nicht die Datenbank. Die Anfragesprache ist meist ein SQL-Dialekt.

2 Datenbankmodelle

Datenbankmodelle entscheiden, wie Daten gespeichert und wie sie abgerufen werden. Jedes der folgenden Datenbankmodelle hat einen eigenen sinnvollen Use Case. Keins der Modelle ist schlecht, trotzdem hat sich heutzutage das relationale Datenbankmodell durchgesetzt.

2.1 Relationales Datenbankmodell

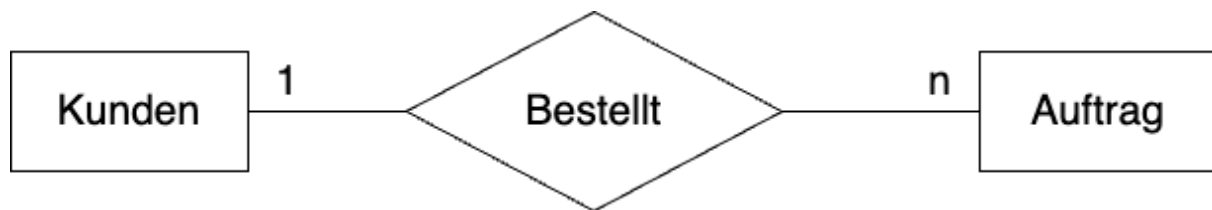


Abbildung 2. Beispiel: relationales Datenbankmodell

Ein relationales Datenbankmodell besteht aus *Relationen*, also Tabellen. Jede Zeile ist ein *Tupel*. Ein Tupel ist eine Liste mit fester Länge, die nicht mehr geändert werden kann. Im Fall des relationalen Datenbankmodells bestehen die Tupel aus einer Reihe von Attributen. Das Schema einer Relation legt dabei die Anzahl und den Typ der Attribute fest. Ein Datensatz in der Tabelle wird `Record` genannt.

Vorteile

- Große Verbreitung
- Einfach und flexibel

Nachteile

- Aufwendige Segmentierung
- Künstliche Schlüsselattribute wie IDs
- Anspruchsvolle Komposition

2.2 Hierarchisches Datenbankmodell

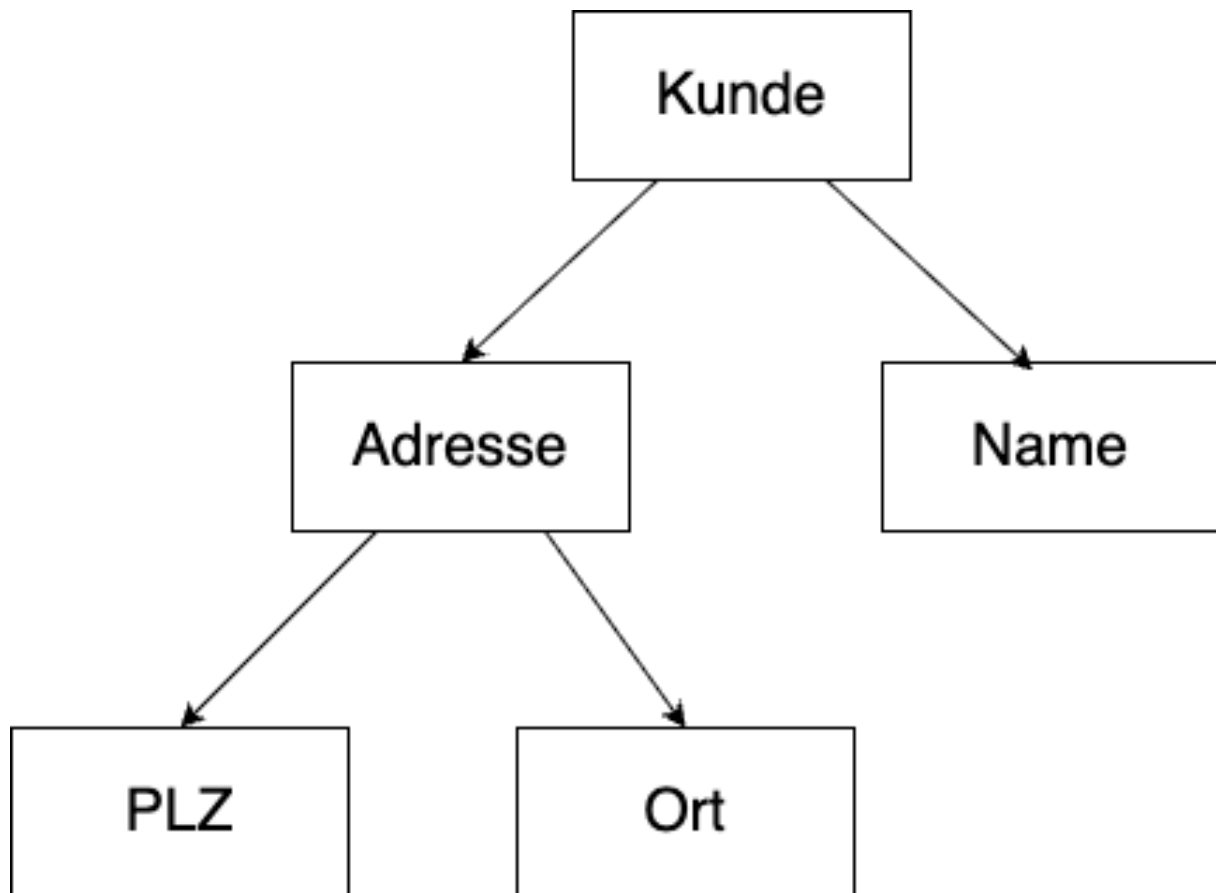


Abbildung 3. Beispiel: hierarchisches Datenbankmodell

Das hierarchische Modell erinnert an Baumstrukturen. Jede Entität hat einen Vorgänger – außer der ersten – und kann mehrere Nachfolger haben. Dadurch ist das Modell gut rekursiv durchsuchbar. Es gibt immer nur einen Einstiegspunkt.

Vorteile

- Effiziente computergerechte Datenorganisation
- Sehr schnell

Nachteile

- Benutzer muss die Datenstruktur gut kennen
- Redundanzen sind möglich

2.3 Netzwerkmodell

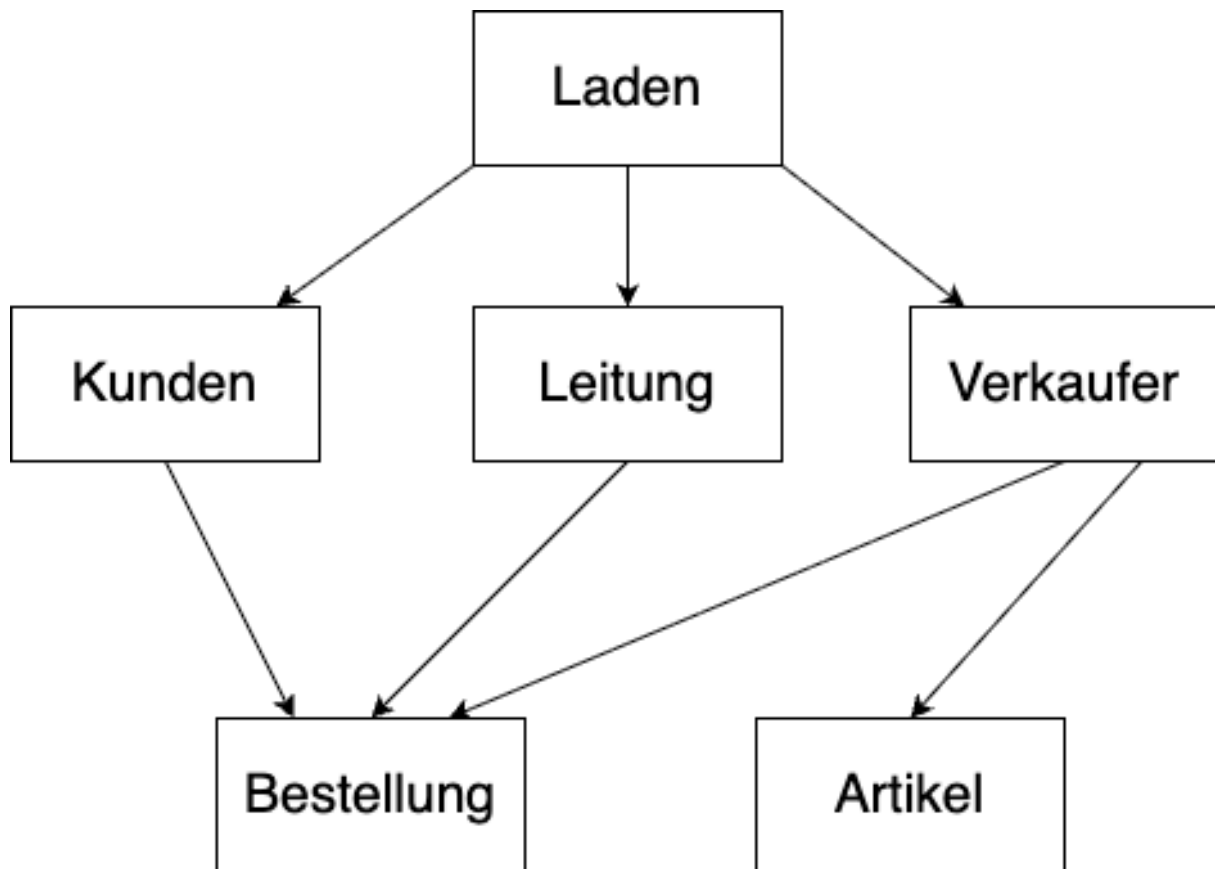


Abbildung 4. Beispiel: Netzwerk-Datenbankmodell

Das Netzwerkmodell ist nah am hierarchischen Modell, allerdings bietet es mehr als einen Einstiegspunkt. So kann man über einen Kunden an dessen Bestellung gelangen, aber auch über den Verkäufer.

Vorteile

- Flexibler Zugriff über mehrere Einstiegspunkte
- Komplexe Strukturen können abgebildet werden

Nachteile

- Benutzer muss die Datenstruktur gut kennen
- Unübersichtlich

2.4 Objektorientiertes Datenbankmodell

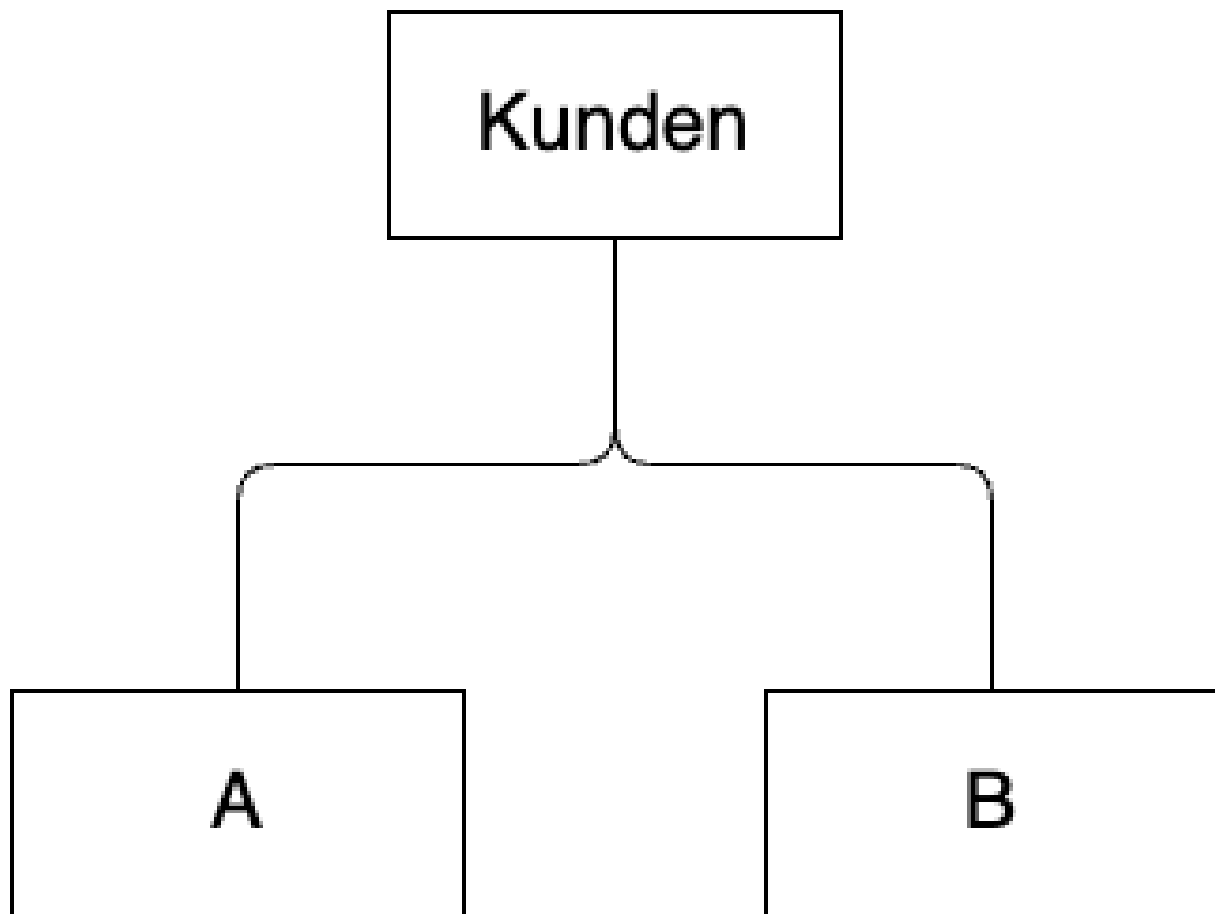


Abbildung 5. Beispiel: objektorientiertes Datenbankmodell

Bei einem objektorientierten Datenbankmodell werden Objekte ohne Zerlegung gespeichert, also als vollständige Objekte. Auch Vererbung, Kapselung und Polymorphie sind integriert. Dadurch lassen sich komplexe Datenstrukturen darstellen.

Heutzutage ist dieses Modell weitgehend obsolet, da es für relationale Datenbanken `ORM`s gibt, z. B. *Entity Framework Core* für C#.

Vorteile

- Das Problem des *Object-Relational Impedance Mismatch* wird behoben
- Komplexe Kompositionen (`JOIN`s) entfallen
- Schlüsselverwaltung systemintern

Nachteile

- Bis heute geringe Verbreitung (z. B. db4o), bei Mengenoperationen langsam
- Aufwändige Bearbeitung vererbter Objekte, inkl. Schreibsperrern

3 Datenverteilung

Logisch zusammenhängende Datenbestände werden physisch verteilt. Die Verwaltung erfolgt dabei übergeordnet durch das Datenbank-System (DB-System).

3.1 Gründe für die Datenverteilung

Die wesentlichen Gründe für die physische Verteilung von logisch zusammenhängenden Daten sind:

- **Verfügbarkeit:** Erhöhung der Systemverfügbarkeit durch Redundanz und lokale Zugriffsmöglichkeiten.
- **Sicherheit:** Schutz der Daten durch Verteilung und kontrollierte Zugriffsmöglichkeiten auf verschiedenen Knoten.
- **Effizienzsteigerungen:** Lokale Verarbeitung von Datenanfragen verringert die Netzwerklast und verbessert die Antwortzeiten.
- **Mobilität:** Unterstützung von mobilen Anwendungen und Zugriffen von verschiedenen geografischen Standorten aus.
- **Skalierbarkeit:** Einfachere Erweiterung der Systemkapazität durch Hinzufügen weiterer Knoten im verteilten System.
- **Zuverlässigkeit:** Erhöhte Ausfallsicherheit durch die Vermeidung von Single Points of Failure.

4 Datenbank-Architektur

4.1 ANSI-3-Ebenen-Modell

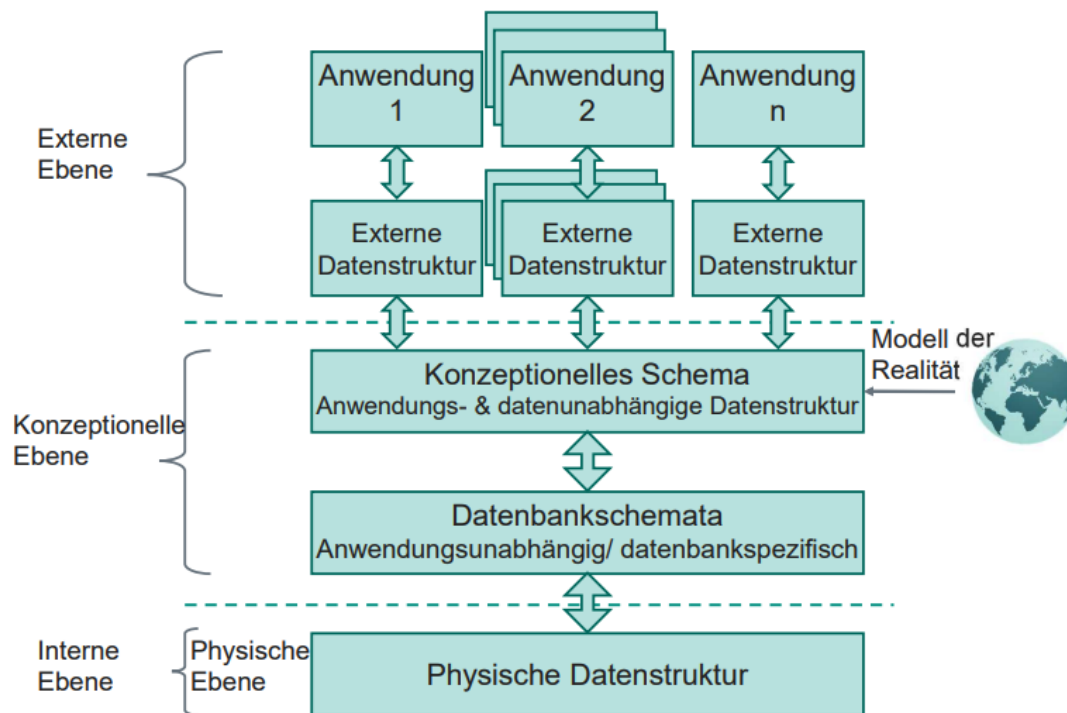


Abbildung 6. Aufbau des ANSI-3-Ebenen-Modells

Die ANSI-3-Ebenen-Architektur gibt es seit den 1970er-Jahren. *ANSI* steht für *American National Standards Institute* und ist damit in etwa mit dem deutschen DIN vergleichbar. Die 3-Ebenen-Architektur wird auch *SPARC* genannt, abgeleitet von *Standards Planning And Requirements Committee* – einem Ausschuss innerhalb von ANSI, von dem die Architektur stammt.

ANSI/SPARC-Architektur

Die 3-Sichten-Architektur ist die Grundlage einer konsistenten, modularen, interoperablen, wartbaren und skalierbaren Datenbank. Sie trennt die Daten in eine **interne**, eine **konzeptionelle** und eine **externe** Ebene.

4.1.1 Interne Ebene

Die interne Ebene ist das Fundament einer Datenbank. Hier liegt ein Datenbankmodell zugrunde, wie es im Kapitel *Datenbankmodelle* besprochen wird. Auf dieser Ebene werden die Aspekte der physischen Speicherung festgelegt – also *wie* und *wo* die Daten konkret abgelegt werden.

4.1.2 Konzeptionelle Ebene

Die konzeptionelle Ebene ist das Bindeglied zwischen der internen und der externen Ebene. Hier werden die Daten modelliert, beispielsweise über ein ER-Diagramm. Auf dieser Ebene wird das Datenbankschema festgelegt, ebenso das konzeptionelle Schema für `Primary Key`, `Foreign Key` sowie sonstige Regeln zur Sicherung der Datenkonsistenz.

4.1.3 Externe Ebene

Die externe Ebene definiert, wie Nutzer auf die Datenbank zugreifen. Dazu zählen User Interfaces, Anwendungs-APIs, Views sowie – im Fall einer API – das Äquivalent in Form von *Data Transfer Objects* (DTOs). Hier können außerdem Plausibilitätsprüfungen und sonstige Integrationsregeln festgelegt werden. Auf dieser Ebene findet der SQL-Zugriff statt.

4.2 Datenbank-Entwurfsphasen

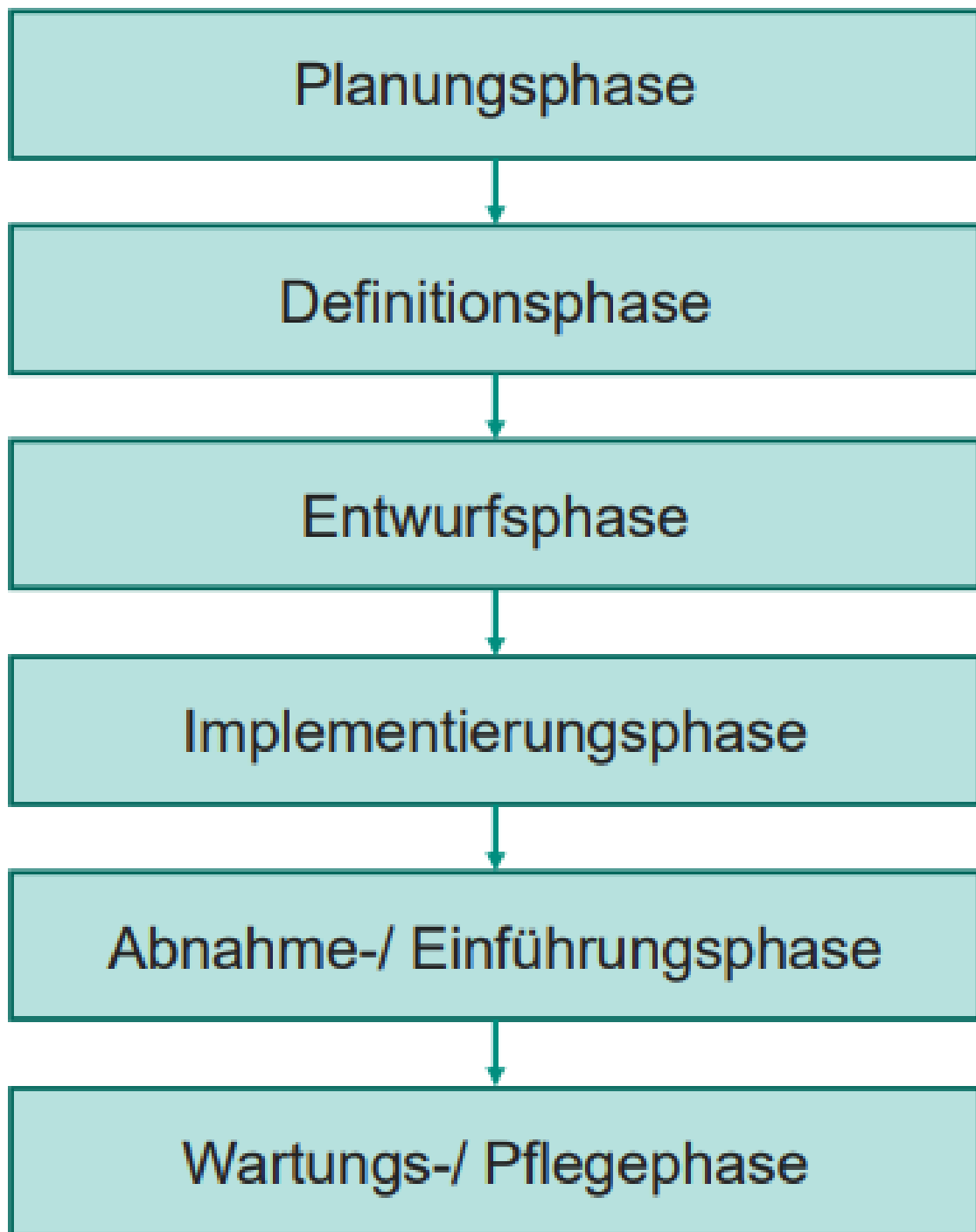


Abbildung 7. Entwurfsphasen einer Datenbank

Der Entwurf einer Datenbank lässt sich in sechs Phasen unterteilen.

4.2.1 Planungsphase

In der Planungsphase geht es um die grundlegenden Festlegungen – also darum, was überhaupt in der Datenbank gespeichert werden soll. Beispielsweise könnte ein Projekt darin bestehen, die Touren eines Logistikunternehmens zu speichern.

4.2.2 Definitionsphase

Hier wird festgelegt, welche Anforderungen die Datenbank erfüllen muss und welche Daten wo und wie erfasst werden müssen.

4.2.3 Entwurfsphase

Hier werden die technischen Entscheidungen getroffen – etwa, welches Datenbankmodell genutzt wird. Außerdem entsteht in dieser Phase das ER-Diagramm.

4.2.4 Implementierungsphase

Hier wird der Entwurf in Code umgesetzt, meist als `DDL` in `SQL`. Eventuell werden auch Views angelegt und vielleicht sogar eine API.

4.2.5 Abnahme- und Einführungsphase

Hier werden die Datenbank bzw. die umgebende Software eingeführt. Anschließend wird geprüft, ob die Anwender mit der Anwendung zufrieden sind.

4.2.6 Wartungs- und Pflegephase

Hier werden mögliche Fehler behoben; auch eventuelle Erweiterungen fallen in diese Phase.

5 Managementunterstützungssysteme

Hinter dem Begriff Managementunterstützungssystem steckt – für alle unter 50 – im Grunde *lobotomized Data Science*, also das Extrahieren von Wissen aus großen Datenmengen, nur ohne Betrachtung der Zukunft und mit einem stärkeren Fokus auf Berichte über vergangene Daten.

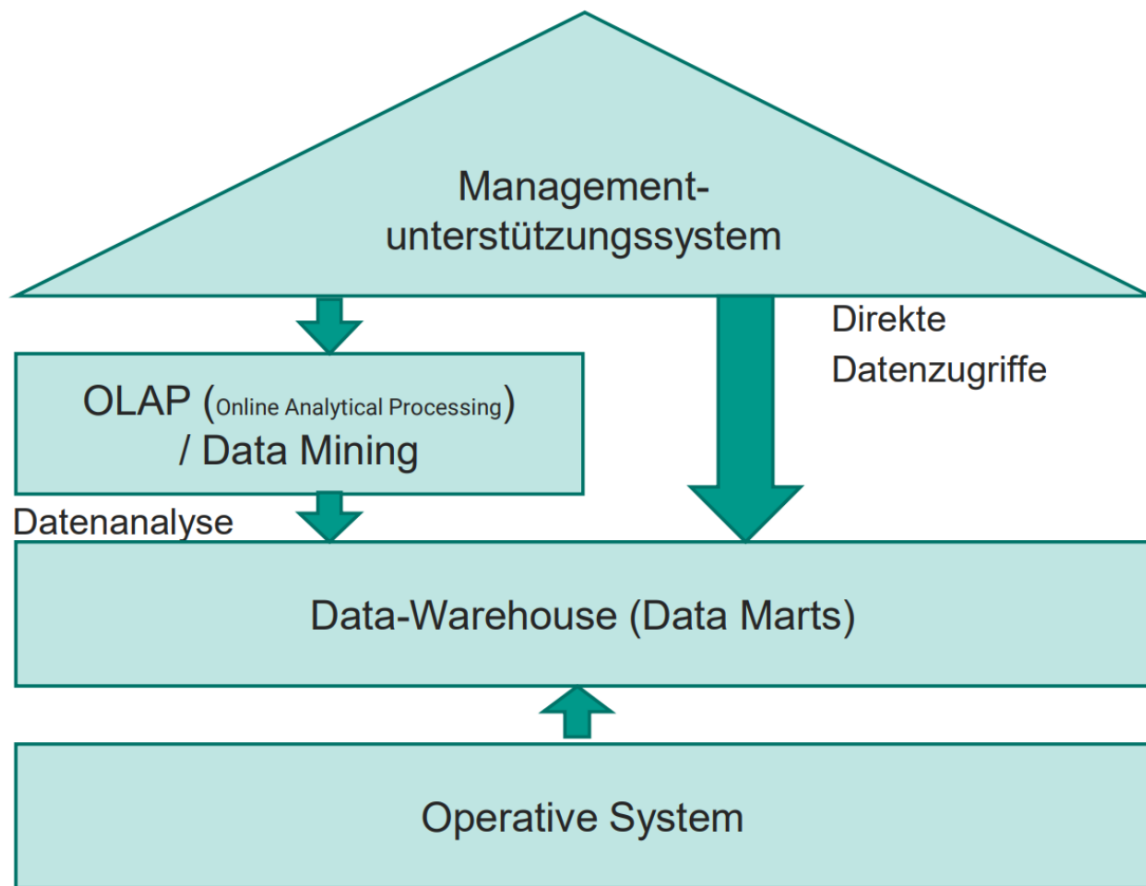


Abbildung 8. Aufbau eines Managementunterstützungssystems

5.1 Data Warehouse

Data Warehousing beschreibt ein Konzept zur Informationsanalyse, -selektion und -aufbereitung, bei dem dem Anwender entscheidungsrelevante Daten aus unterschiedlichen Quellen in einer einheitlichen, zentralen Systemumgebung zur Auswertung zur Verfügung gestellt werden.

- Ein Data Warehouse ist eine extrem große Datenbank, die unabhängig von den operativen Systemen installiert ist und ein vordefiniertes Set von meist verdichteten Daten enthält.
- Die Benutzer können auf die Datenbank beliebig zugreifen und alle gewünschten Daten herausziehen, sofern die Informationen enthalten sind und eine Zugriffsberechtigung vorliegt.
- Der Grad der Informationstiefe und die jeweilige Sichtweise können dabei individuell festgelegt werden.
- Betrachtungsebenen sind beispielsweise Konten, Kostenstellen, Kostenträger, Artikel, Kunden, Märkte oder Vertreter. Es gibt keine festen Vorgaben für die Art der Reihenfolge und der Verdichtung.

5.2 Data Mart

Data Marts sind abteilungs- oder funktionsbezogene Datenbanken mit einer speziellen Aufgabenstellung, die eine Untermenge der im zentralen Data Warehouse gespeicherten

operativen Daten enthalten.

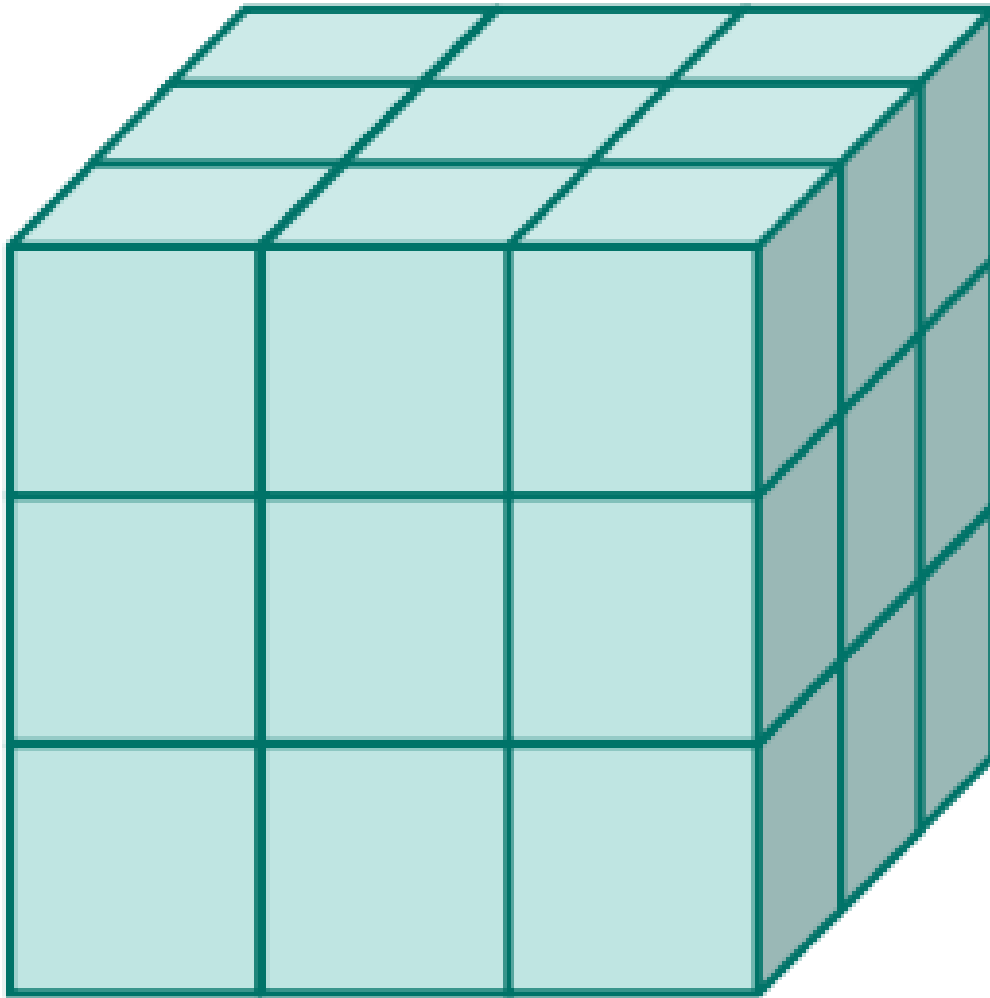


Abbildung 9. Data Mart

- Data Marts enthalten lediglich Informationen, die in einem bestimmten Funktionsbereich, z. B. Marketing, des Unternehmens zur Anwendung kommen.
- Data Marts bilden daher eine Teilmenge der umfassenden Data-Warehouse-Datenbank.
- Data Marts werden auch Datenwürfel, Power Cube oder Info Cube genannt.

5.3 Online Analytical Processing – OLAP

Online Analytical Processing ist ein „top-down“-Ansatz, um sehr flexibel mehrdimensionale Datenanalysen durchzuführen und Hypothesen zu bilden.

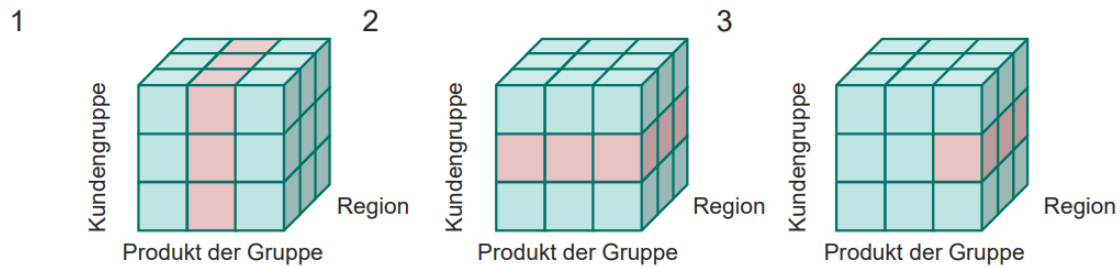


Abbildung 10. Datenanalyse mit OLAP

5.4 Data Mining

Data Mining bezeichnet das automatische Entdecken von Modellen und Mustern in großen Datenbeständen durch Anwendung bestimmter Data-Mining-Werkzeuge.

- Data Mining ist eine „intelligente“ Anwendung auf Basis einer Data-Warehouse-Architektur.
- Beziehungen zwischen Daten werden hergestellt (Zeitreihenanalyse, Datenklassifikation, Prognosen).
- Data Mining versucht, den Anwender auf bemerkenswerte Datenkonstellationen aufmerksam zu machen und zu signifikanten Aussagen hinzuführen.
- Analyseziel: „Finde Gold in deinen Daten!“